

Proyecto Aplicado de Cálculo y Analítica Empresarial

Modelado, Optimización y Desarrollo de Herramientas en Python

Límites · Derivadas · Optimización · Análisis Marginal · Software Empresarial

Semanas 2 a 7 | Trabajo en parejas | Valor: de la nota total

I. Descripción General

Pregunta Central del Proyecto

“¿Cómo puede una solución de software empresarial, sustentada en modelos de cálculo diferencial, transformar datos operativos en recomendaciones para la toma de decisiones?”

Este proyecto integra cálculo diferencial, análisis cuantitativo, programación en Python y necesidades propias de la **Informática Empresarial**. Cada pareja desarrollará un **prototipo analítico** para apoyar una decisión de una organización.

El prototipo puede ser una aplicación de consola, un script modular o un notebook ejecutable en Google Colab. Debe recibir datos o parámetros, ejecutar el modelo matemático, generar tablas y gráficas, y comunicar una recomendación comprensible para una persona no técnica.

El proyecto no consiste únicamente en derivar una función ni en mostrar una gráfica. Debe responder una necesidad empresarial concreta, como fijación de precios, planificación de capacidad tecnológica, control de inventario, análisis de rentabilidad, estimación de crecimiento digital o comparación de alternativas financieras.

Datos Logísticos

- **Modalidad:** Trabajo en parejas, máximo dos integrantes.
- **Producto final:** Informe en PDF, código Python y datos o generador de datos.
- **Valor:** 10 % de la nota total del curso.
- **Extensión:** Entre 8 y 12 páginas, sin incluir portada, referencias ni anexos.
- **Defensa oral:** Obligatoria para ambos integrantes.
- **Tema:** Cada pareja seleccionará uno de los cinco proyectos y lo registrará con el docente.

Enfoque de Informática Empresarial

Cada proyecto debe simular una solución que pueda ser utilizada por una empresa o institución. Por ello, todo trabajo debe definir:

- una **persona usuaria objetivo**: gerencia, finanzas, mercadeo, ventas, operaciones, inventario o administración;
- una **decisión empresarial**: precio, capacidad, producción, inventario, inversión, campaña o presupuesto;
- una **entrada de datos**: CSV, tabla simulada, parámetros ingresados por teclado o formulario sencillo;
- una **salida útil**: tabla, gráfica, alerta, recomendación, indicador o comparación de escenarios;
- una explicación clara de cómo el cálculo diferencial contribuye a mejorar la decisión.

Uso de Inteligencia Artificial y Defensa Oral

Se permite utilizar herramientas de Inteligencia Artificial para apoyar la búsqueda bibliográfica, la organización de ideas, la corrección de estilo, la depuración de código y el diseño de pruebas. Sin embargo, la IA no sustituye la comprensión matemática ni técnica.

Durante la defensa oral, ambos integrantes deberán explicar:

- el problema empresarial;
- el origen de los datos o los supuestos utilizados;
- las variables, unidades y restricciones;
- los límites, derivadas, extremos o cantidades marginales calculadas;
- la lógica de las funciones programadas;
- las gráficas y la interpretación de resultados;
- la recomendación final.

Condición obligatoria: si una pareja no puede explicar partes fundamentales del informe, del modelo matemático o del código Python, la calificación podrá ser de **0 (cero)**. El uso de IA no será penalizado por sí mismo; se evaluará la comprensión, autenticidad y capacidad de defensa.

II. Relación con los Contenidos del Curso

Contenidos de las semanas 2 a 7

Semana	Contenidos aplicables al proyecto
2	Concepto de límite. Límites por factorización, racionalización y cambio de variable.
3	Propiedades de límites, valor absoluto, límites al infinito, límites infinitos, continuidad y discontinuidades.
4	Derivada como pendiente, diferencial, razón de cambio, reglas de derivación, orden superior, diferenciabilidad, regla de la cadena y derivación implícita.
5	Crecimiento y decrecimiento, máximos y mínimos, criterios de la primera y segunda derivada, concavidad y representación de curvas.
6	Problemas de máximos y mínimos y aplicaciones de la derivada al análisis marginal.
7	Costo marginal, ingreso marginal, utilidad marginal, producto marginal, impuesto marginal e interés compuesto.

Requisito matemático común

Independientemente del tema elegido, cada proyecto debe incluir:

- al menos una función definida dentro de un contexto real;
- una derivada calculada analíticamente e interpretada;
- un análisis de crecimiento/decrecimiento o una razón de cambio;
- un punto crítico, extremo, comparación de escenarios o meta cuantificable;
- una gráfica generada en Python;
- una recomendación empresarial sustentada en los resultados.

III. Proyectos Aplicados

Proyecto 1 — Tarifas por Tramos y Demanda de un Servicio Digital

Rol empresarial: Analista de mercadeo y ventas.

Contexto. Una empresa ofrece un servicio digital: almacenamiento en la nube, tutorías virtuales, plataforma de cursos, suscripciones de contenido o soporte técnico. La tarifa cambia según el nivel de uso, la promoción vigente o el tipo de cliente.

Problema empresarial. La empresa necesita recomendar una tarifa que maximice el ingreso esperado sin disminuir excesivamente la cantidad de clientes.

Pregunta de decisión:

¿Qué precio o tarifa maximiza el ingreso esperado, considerando una promoción, descuento o cambio de demanda en un precio crítico?

Modelo mínimo requerido. Construyan una función de demanda $D(p)$ dependiente del precio p . Debe incluir una condición por tramos o valor absoluto. Por ejemplo:

$$D(p) = \begin{cases} D_1(p), & p < p_c, \\ D_2(p), & p \geq p_c, \end{cases}$$

donde p_c es un precio crítico asociado a un descuento, promoción o cambio de tarifa. Definan el ingreso:

$$R(p) = pD(p).$$

Conceptos de cálculo obligatorios:

- Dos límites laterales en $p = p_c$.
- Análisis de continuidad o discontinuidad de $D(p)$ y/o $R(p)$.
- Derivada $R'(p)$.
- Intervalos de crecimiento y decrecimiento del ingreso.
- Punto crítico y determinación del precio que maximiza el ingreso.
- Interpretación del resultado dentro de un intervalo de precios viable.

Requisitos de Python:

- Implementar `demanda(precio)`, `ingreso(precio)` y `recomendar_precio(...)`.
- Utilizar un CSV con precios y demanda estimada o un generador de datos simulados documentado.
- Graficar $D(p)$ y $R(p)$.
- Señalar en la gráfica el precio crítico y el precio recomendado.
- Generar una tabla con al menos 15 valores de precio.
- Comparar los escenarios: precio bajo, precio recomendado y precio alto.
- Emitir un mensaje automático que indique el precio recomendado, demanda estimada e ingreso esperado.

Producto final esperado. Un prototipo de recomendación de tarifas para un encargado de mercadeo o ventas.

Proyecto 2 — Capacidad de Servidores y Costo Marginal

Rol empresarial: Analista de operaciones tecnológicas o infraestructura.

Contexto. Una empresa de software brinda una plataforma web. A medida que aumentan los usuarios simultáneos, incrementan los costos de servidores, almacenamiento, soporte técnico y monitoreo. La empresa debe decidir qué capacidad es económicamente conveniente.

Problema empresarial. La empresa necesita estimar cuántos usuarios simultáneos puede soportar antes de que los costos crezcan más rápido que los beneficios.

Pregunta de decisión:

¿Cuántos usuarios simultáneos debe soportar la plataforma para maximizar la utilidad y en qué momento se recomienda ampliar la infraestructura?

Modelo mínimo requerido. Sea x la cantidad de usuarios simultáneos. Definan:

$C(x)$ = costo de infraestructura,

$R(x)$ = ingreso generado por los usuarios,

$U(x) = R(x) - C(x)$.

Debe definirse una capacidad máxima física o contractual de la plataforma.

Conceptos de cálculo obligatorios:

- Costo marginal $C'(x)$.
- Ingreso marginal $R'(x)$.
- Utilidad marginal $U'(x)$.
- Punto crítico de $U(x)$.
- Criterio de la primera o segunda derivada para clasificar el extremo.
- Concavidad de la utilidad o del costo.
- Análisis de $x \rightarrow \infty$ para explicar por qué la capacidad no puede crecer indefinidamente.

Requisitos de Python:

- Implementar `costo(x)`, `ingreso(x)`, `utilidad(x)` y `recomendar_capacidad(...)`.
- Permitir modificar costos fijos, costo por usuario, tarifa y capacidad máxima.
- Calcular y mostrar $C'(x)$, $R'(x)$ y $U'(x)$.
- Graficar costo, ingreso y utilidad.
- Marcar el valor de x que maximiza la utilidad.
- Comparar infraestructura básica, intermedia y ampliada.
- Generar una alerta cuando la demanda proyectada supere la capacidad máxima.

Producto final esperado. Un prototipo de planificación de capacidad para apoyar al área de tecnología y operaciones.

Proyecto 3 — Inventario Óptimo y Utilidad Neta de una Tienda Virtual

Rol empresarial: Analista financiero, de inventario o comercio electrónico.

Contexto. Una tienda virtual vende un producto con costo fijo, costo variable, precio de venta, descuentos por cantidad y capacidad limitada de almacenamiento. La administración necesita decidir cuántas unidades comprar o producir por periodo.

Problema empresarial. La tienda requiere establecer una cantidad de inventario que maximice la utilidad después de costos e impuestos.

Pregunta de decisión:

¿Qué cantidad de inventario maximiza la utilidad neta después de considerar costos, precio, capacidad, descuentos e impuesto marginal?

Modelo mínimo requerido. Sea q la cantidad de unidades. Definan:

$$C(q) = \text{costo total}, \quad R(q) = \text{ingreso total}, \quad U(q) = R(q) - C(q).$$

Incluyan un impuesto por tramos o una tasa de impuesto marginal:

$$U_N(q) = U(q) - T(U(q)).$$

Conceptos de cálculo obligatorios:

- Punto de equilibrio.
- Costo marginal $C'(q)$.
- Ingreso marginal $R'(q)$.
- Utilidad marginal $U'(q)$.
- Utilidad neta e impuesto marginal.
- Crecimiento, decrecimiento y concavidad.
- Cantidad que maximiza la utilidad neta.
- Restricción de inventario máximo.

Requisitos de Python:

- Implementar `costo(q)`, `ingreso(q)`, `utilidad(q)`, `impuesto(q)` y `recomendar_inventario(...)`.
- Cargar un CSV de inventario y ventas, o generar datos simulados justificados.
- Graficar costo, ingreso, utilidad antes de impuestos y utilidad neta.
- Calcular puntos de equilibrio y cantidad óptima.
- Crear tabla con al menos 15 niveles de inventario.
- Comparar tres escenarios de precio, costo variable o impuesto.
- Generar una recomendación de inventario para el siguiente periodo.

Producto final esperado. Un tablero de rentabilidad para apoyar decisiones de compras, inventario y ventas.

Proyecto 4 — Crecimiento de Usuarios y Planificación de Capacidad Digital

Rol empresarial: Analista de datos, producto digital o mercadeo.

Contexto. Una plataforma digital registra usuarios activos por semana o por mes. La organización necesita anticipar cuándo ampliar servidores, soporte técnico, almacenamiento o presupuesto de mercadeo.

Problema empresarial. Se debe estimar el comportamiento futuro de usuarios activos y reconocer el momento en que la capacidad del sistema será insuficiente.

Pregunta de decisión:

¿En qué periodo se debe ampliar la infraestructura según el crecimiento, la aceleración de usuarios y la capacidad máxima de la plataforma?

Modelo mínimo requerido. Construyan un modelo de usuarios activos:

$N(t)$ = número de usuarios activos en el tiempo t .

Puede ser una función exponencial, logística o por tramos. Si se usan datos simulados, deben justificarse los parámetros.

Conceptos de cálculo obligatorios:

- Límite de $N(t)$ cuando $t \rightarrow \infty$.
- Derivada $N'(t)$ como tasa de crecimiento.
- Segunda derivada $N''(t)$ como aceleración o desaceleración.
- Intervalos de crecimiento y concavidad.
- Punto de inflexión, si el modelo lo permite.
- Periodo en el cual se alcanza la capacidad máxima.
- Discusión sobre la viabilidad de proyectar crecimiento indefinido.

Requisitos de Python:

- Utilizar al menos 20 periodos de datos reales con fuente o datos simulados reproducibles.
- Implementar `usuarios(t)`, `tasa_crecimiento(t)`, `aceleracion(t)` y `proyectar_capacidad(...)`.
- Graficar $N(t)$, $N'(t)$ y $N''(t)$.
- Marcar la capacidad máxima y el periodo estimado de saturación.
- Generar una alerta automática de ampliación de infraestructura.
- Comparar al menos dos escenarios de crecimiento.
- Mostrar una tabla de proyección semanal o mensual.

Producto final esperado. Un prototipo de analítica de crecimiento para la planificación de capacidad tecnológica.

Proyecto 5 — Simulador de Ahorro

Rol empresarial: Analista financiero, desarrollador de software financiero o asesor de productos de ahorro.

Contexto. Una entidad financiera o una aplicación de finanzas personales desea ofrecer una herramienta para comparar planes de ahorro, inversión o pago de deuda.

Problema empresarial. La aplicación debe ayudar a una persona a comparar alternativas financieras y determinar cuál le permite alcanzar una meta con menor tiempo, mayor rendimiento o menor aporte inicial.

Pregunta de decisión:

¿Qué alternativa financiera permite alcanzar una meta con menor plazo o menor aporte y cuáles variables tienen mayor impacto en el resultado?

Modelo mínimo requerido. Deben comparar interés simple, interés compuesto y capitalización continua.

Interés compuesto:

$$A(t) = P \left(1 + \frac{r}{n} \right)^{nt}.$$

Donde:

P = capital inicial, r = tasa anual, n = capitalizaciones por año, t = tiempo.

Conceptos de cálculo obligatorios:

- Funciones exponenciales.
- Límite al infinito del capital acumulado.
- Derivada del monto respecto del tiempo.
- Razón de crecimiento del capital.
- Diferenciales o sensibilidad con respecto a la tasa r o capital inicial P .
- Cálculo del tiempo para alcanzar una meta.
- Comparación de escenarios de tasa, plazo y capitalización.

Requisitos de Python:

- Implementar `interes_simple(...)`, `interes_compuesto(...)`, `capitalizacion_continua(...)` y `tiempo_para_meta(...)`.
- Solicitar o definir capital inicial, tasa, plazo, meta y frecuencia de capitalización.
- Generar una tabla comparativa de al menos tres alternativas.
- Graficar el crecimiento del capital para cada alternativa.
- Graficar el efecto de modificar la tasa de interés.
- Calcular una medida de sensibilidad ante un cambio pequeño de la tasa.
- Emitir una recomendación automática según la meta definida.

Producto final esperado. Un simulador financiero para apoyar decisiones de ahorro o inversión.

IV. Estructura Obligatoria del Informe

Secciones del Informe

1. **Portada:** Universidad, curso, título, nombres completos, carnés, grupo, docente y fecha.
 2. **Introducción:** Contexto empresarial, importancia del problema, usuario objetivo y pregunta de decisión.
 3. **Marco teórico:** Conceptos de cálculo y programación utilizados. Debe incluir al menos un procedimiento matemático desarrollado completamente a mano.
 4. **Datos y supuestos:** Fuente de los datos o justificación de datos simulados; variables, unidades, dominio, restricciones y limitaciones.
 5. **Modelo matemático:** Funciones definidas, límites, derivadas, puntos críticos, análisis marginal, interés compuesto u otros procedimientos pertinentes.
 6. **Desarrollo y resultados:** Tablas, cálculos, gráficas, escenarios y validación del modelo.
 7. **Implementación en Python:** Descripción del programa, funciones principales, evidencia de ejecución, visualizaciones y explicación de resultados.
 8. **Conclusiones y recomendación empresarial:** Respuesta a la pregunta de decisión, recomendación, limitaciones y posibles mejoras.
 9. **Referencias bibliográficas:** Mínimo cuatro fuentes en formato APA 7.
 10. **Anexos:** Código completo, CSV u otros datos, y cálculos extensos.
- Extensión:** mínimo 8 páginas y máximo 12 páginas, sin incluir portada, referencias ni anexos. Utilice Times New Roman 12 o equivalente e interlineado 1.5.

Validación obligatoria del modelo

Todo resultado debe ser interpretado dentro del contexto empresarial. Cada pareja debe discutir:

- el dominio válido de las variables;
- las unidades y magnitudes utilizadas;
- restricciones de capacidad, presupuesto, demanda, inventario o mercado;
- coherencia entre cálculo manual, resultados de Python y gráficas;
- al menos tres limitaciones del modelo;
- qué datos adicionales mejorarían la recomendación.

Un resultado correcto algebraicamente, pero imposible o poco realista en el contexto, no puede presentarse como recomendación sin una discusión crítica.

V. Requisitos de Python y Reproducibilidad

Requisitos Técnicos del Código

- El programa debe ejecutarse sin errores en **Python 3.10+** o Google Colab.
- Se permite utilizar **numpy**, **pandas**, **matplotlib**, **sympy** y **scipy**.
- El código debe incluir al menos **tres funciones propias**: una para el modelo, una para un indicador derivativo o marginal y una para gráficos, escenarios o recomendaciones.
- Debe leer datos desde CSV o generar datos simulados mediante una semilla y parámetros documentados.
- Debe generar al menos una tabla, dos gráficas y una recomendación textual.
- Debe permitir modificar al menos dos parámetros del modelo sin alterar la lógica principal.
- Debe verificar al menos un resultado analítico con una evaluación numérica o procedimiento computacional.
- Debe incluir comentarios y nombres de variables comprensibles.
- Debe entregarse como archivo **.py** o enlace público y funcional a Google Colab.

Estructura recomendada de un prototipo analítico

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # 1. Funcion del modelo empresarial
6 def modelo(x, parametros):
7     return ...
8
9 # 2. Indicador de calculo:
10 # derivada, costo marginal, utilidad marginal, etc.
11 def indicador(x, parametros):
12     return ...
13
14 # 3. Funcion para analizar escenarios y recomendar
15 def recomendar(parametros):
16     return {
17         "decision": ...,
18         "indicador": ...,
19         "mensaje": "Recomendacion generada por el modelo."
20     }
21
22 # Parametros modificables
23 parametros = {
24     "precio": 10000,
25     "costo_variable": 5000
26 }
27
28 x = np.linspace(0, 100, 200)
29 y = modelo(x, parametros)
30
31 resultado = recomendar(parametros)
32 print(resultado["mensaje"])
33
34 plt.figure(figsize=(8, 5))
35 plt.plot(x, y, label="Modelo")
36 plt.xlabel("Variable independiente")
37 plt.ylabel("Indicador empresarial")
38 plt.title("Análisis del modelo")
39 plt.grid(True, linestyle="--", alpha=0.5)
40 plt.legend()
41 plt.show()

```

Datos reales y datos simulados

Se permiten datos reales o simulados.

- **Datos reales:** Deben incluir fuente, fecha de consulta, unidades y referencia bibliográfica.
- **Datos simulados:** Deben incluir fórmula o procedimiento de generación, parámetros, semilla cuando corresponda y justificación de por qué son razonables.

No se permite presentar datos simulados como si fueran observaciones reales.

VI. Avances, Exposición y Defensa

Hitos del Proyecto

Momento	Entregable	Propósito
Inicio de semana 5	Tema, integrantes y pregunta de decisión	Validar alcance, relación con el curso y evitar repetición de proyectos.
Semana 6	Avance de una página: variables, datos, supuestos, modelo preliminar y una gráfica inicial	Recibir retroalimentación antes de construir el prototipo completo.
Semana 7	Informe final, código, datos y presentación	Evaluación final del proyecto.

Defensa Oral

Cada pareja tendrá entre **10 y 12 minutos** para exponer. Ambos integrantes deben participar y poder explicar:

- el problema y la decisión empresarial;
- las variables, supuestos y restricciones;
- al menos una derivada o procedimiento de cálculo;
- las funciones programadas;
- las gráficas obtenidas;
- la recomendación final.

VII. Rúbrica de Evaluación

Rúbrica Analítica		
Criterio	Descripción	Puntos
Problema empresarial y usuario objetivo	Contexto claro, decisión cuantificable y usuario objetivo definido.	10
Datos, supuestos y validación	Datos trazables o simulación justificada, dominio, unidades, restricciones y limitaciones.	10
Modelo matemático y cálculo	Funciones, límites, derivadas, extremos, concavidad, análisis marginal o interés compuesto correctamente aplicados.	20
Prototipo en Python	Código funcional, modular, reproducible, con lectura/generación de datos, funciones y escenarios.	20
Gráficas, escenarios e interpretación	Visualizaciones claras, comparación de escenarios y recomendación sustentada.	15
Informe, conclusiones y referencias	Estructura requerida, redacción, conclusiones, limitaciones y APA 7.	10
Defensa individual	Dominio matemático, técnico y contextual demostrado por cada integrante.	15
Total		100

Nota grupal e individual

La nota del informe, modelo, código y visualizaciones corresponde al trabajo de la pareja. La nota de defensa oral puede ser diferente para cada integrante según su participación y dominio demostrado.

VIII. Advertencias Importantes

► Condiciones obligatorias

1. **Proyecto no defendible:** Si no pueden explicar el modelo, los cálculos o el código, la nota puede ser 0.
2. **Código no ejecutable:** Si el programa no corre y no existe evidencia reproducible, el criterio de programación recibirá puntaje mínimo.
3. **Datos sin fuente:** Toda tabla, imagen, dato real o conjunto externo debe citarse correctamente.
4. **Resultados sin interpretación:** No basta con mostrar fórmulas o gráficas; deben explicar la decisión empresarial recomendada.
5. **Datos simulados sin justificación:** Los datos simulados deben declararse y justificarse; no se pueden presentar como reales.
6. **Participación insuficiente:** Ambos integrantes deben participar en la presentación y defensa del proyecto.

“El valor del cálculo no está solo en derivar una función, sino en usar sus resultados para tomar mejores decisiones.”
